

STATSIG lecture 11

Adaptive filters 1 **LMS algorithm**

Disclaimer:

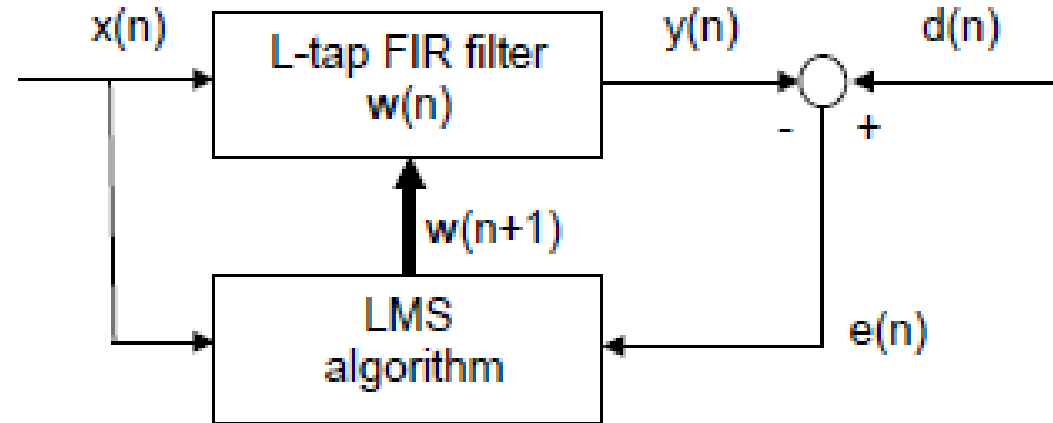
**Please note that these slides are only a brief overview of the topic.
Many more details will be explained on the blackboard and with
MATLAB examples**

Adaptive filters and LMS algorithm

Applicable for non-stationary (not LTI) systems and signals.

Is specified in terms of cost function (MSE) in time-domain and not like standard digital filters in terms of cutoff frequencies.

Generic model
(detailed definitions
on blackboard)



input signal $x(n)$

standard FIR-filter with L coefficients in vector $w(n)$

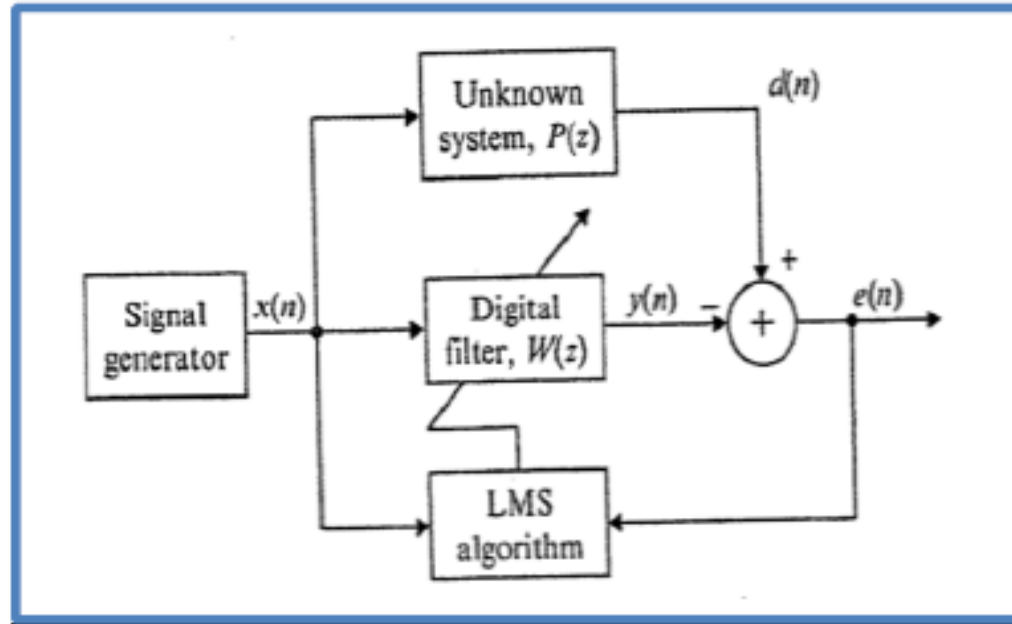
output from filter $y(n)$

"desired signal" $d(n)$ (generated in different ways dependent on actual application)

error signal $e(n) = d(n) - y(n)$

adaptive LMS algorithm calculates filter coefficients $w(n+1)$ for next sample period

Application: Adaptive system identification of time-varying system
Other applications follows later in lecture



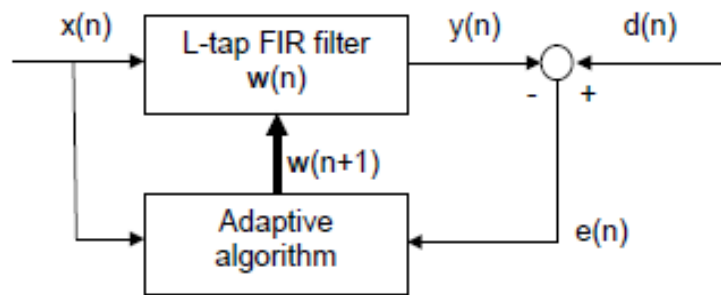
Domain:

Acoustics, Vibration, Communication, Control, ...

Operation:

Adaptive algorithm minimizes $e(n) \Rightarrow W(z) \cong P(z)$

Step 1. Optimal adaptive FIR filter – Wiener Hopf reviewed



Cost-function

$$\begin{aligned}\xi(n) &= E[e^2(n)] = E\left[\left(d(n) - \mathbf{w}^T(n)\mathbf{x}(n)\right)^2\right] \\ &= E[d^2(n)] - 2\mathbf{w}^T(n)E[d(n)\mathbf{x}(n)] + \mathbf{w}^T(n)E[\mathbf{x}(n)\mathbf{x}^T(n)]\mathbf{w}(n)\end{aligned}$$

Autocorrelation matrix

$$\mathbf{R}_{xx}(n) = E[\mathbf{x}(n)\mathbf{x}^T(n)] = \begin{bmatrix} r_{xx}(0) & \cdots & r_{xx}(L-1) \\ \vdots & \ddots & \vdots \\ r_{xx}(L-1) & \cdots & r_{xx}(0) \end{bmatrix}$$

Cross correlation vector

$$\mathbf{r}_{dx}(n) = E[d(n)\mathbf{x}(n)] = \begin{bmatrix} r_{dx}(0) \\ \vdots \\ r_{dx}(L-1) \end{bmatrix}$$

Giving

$$\xi(n) = E[d^2(n)] - 2\mathbf{w}^T(n)\mathbf{r}_{dx}(n) + \mathbf{w}^T(n)\mathbf{R}_{xx}(n)\mathbf{w}(n)$$

Optimization

$$\nabla \xi(n) = \frac{\partial \xi}{\partial \mathbf{w}} = 0 - 2\mathbf{r}_{dx}(n) + 2\mathbf{R}_{xx}(n)\mathbf{w}(n) = \mathbf{0}$$

Optimal filter (Wiener Hopf solution)

$$\mathbf{w}(n) = \mathbf{w}_{opt}(n) = \mathbf{R}_{xx}^{-1}(n) \mathbf{r}_{dx}(n)$$

However, for non-stationary signals, the **optimal filter coefficients must be recalculated frequently** (perhaps even for every sample period)

To do so, **the autocorrelation matrix and the crosscorrelation vector must be estimated over and over again** and this has a **very high computational complexity** (details on blackboard)

Instead, an **iterative search towards optimal filter coefficients can be applied**

Step 2. Steepest descent iteration towards optimal filter coefficients

The cost function decreases at most in the opposite direction of it's gradient (details on blackboard)

This gives $w(n+1) = w(n) - \frac{\mu}{2} \nabla \xi = w(n) - \mu [R_{xx}(n)w(n) - r_{dx}(n)]$

This has lower computational complexity than Wiener-Hopf but still high

Step 3: LMS algorithm

Redefinition of cost function $\xi(n) = E[e^2(n)] \rightarrow \xi(n) = e^2(n)$

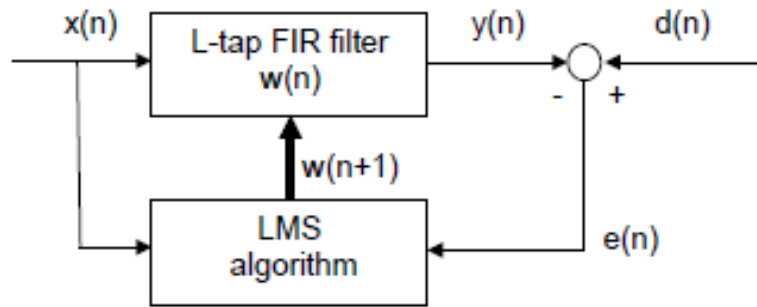
Removing the $E[\]$ expectation removes the autocorrelation matrix and crosscorrelation vector

Giving finally **LMS algorithm** (details and derivation on blackboard)

$$w(n+1) = w(n) + \mu e(n) x(n)$$

which has low computational complexity

LMS organized as an algorithm



<u>Parameters</u>		$L = \text{filter length}$ $\mu = \text{step size}$
<u>Init</u>		$\mathbf{x}(0) = \begin{bmatrix} x(0) \\ \vdots \\ x(-(L-1)) \end{bmatrix} = \mathbf{0}$ $\mathbf{w}(0) = \begin{bmatrix} w_0(0) \\ \vdots \\ w_{L-1}(0) \end{bmatrix} = \mathbf{0}$ $n = 1$
<u>Loop</u>	<u>Get data</u>	$\mathbf{x}(n) = \begin{bmatrix} x(n) \\ \vdots \\ x(n-(L-1)) \end{bmatrix}$ $d(n)$
	<u>Predict</u>	$\hat{d}(n) = y(n) = \mathbf{w}^T(n-1) \mathbf{x}(n)$
	<u>Error</u>	$e(n) = d(n) - \hat{d}(n)$
	<u>Update filter</u>	$\mathbf{w}(n) = \mathbf{w}(n-1) + \mu e(n) \mathbf{x}(n)$
	<u>Iterate</u>	$n = n + 1$
<u>End</u>		

Computational complexity $\mathcal{O}(L)$

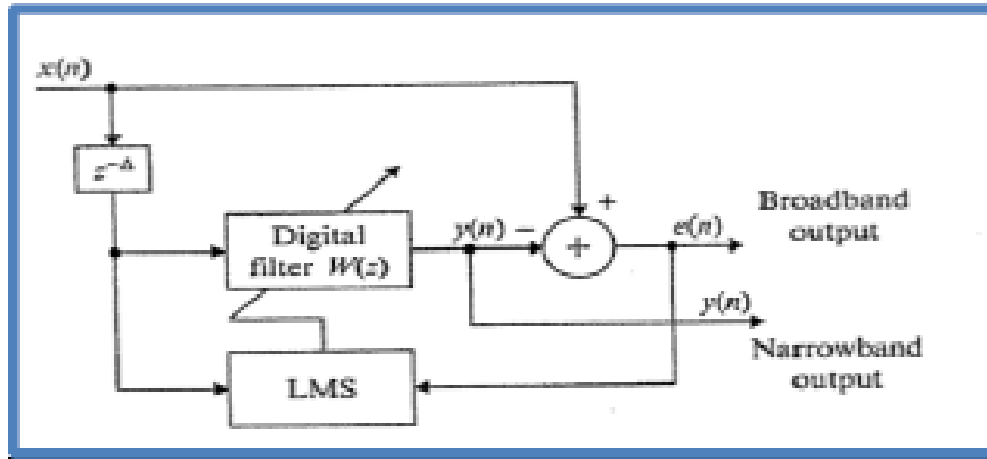
Variants of LMS (details on blackboard): Normalized LMS (timevarying stepsize)
Leaky LMS (with "forgetting" factor)

LMS algorithm performance measures

(blackboard, pdf-file "LMS algorithm and performance", MATLAB demos)

Application: Adaptive Linear Prediction

Separation of narrowband signal parts (classically the actual signal) and wideband signal parts (classically wideband ("white") noise)



Domain:

Communications, ...

Operation:

Signal $x(n) = s(n) + v(n)$ consists of narrowband part, $s(n)$, and wideband part, $v(n)$

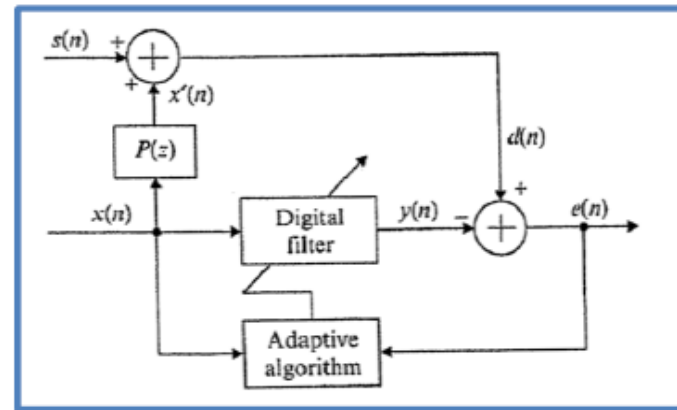
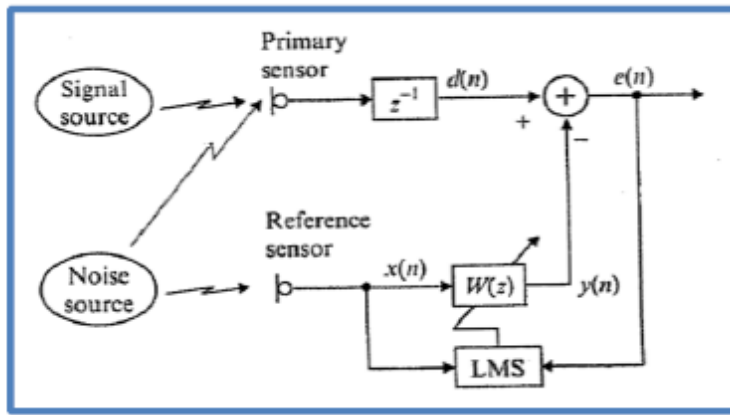
Adaptive algorithm minimizes $e(n)$ by linear prediction of $x(n)$ from $x(n - \Delta)$

The delay, Δ , is chosen big enough to ensure $R_{vv}(k) \cong 0$ for $|k| \geq \Delta$ (e.g. WN: $\Delta = 1$)

Then $v(n)$ is uncorrelated with $v(n - \Delta)$, but $s(n)$ is correlated with $s(n - \Delta)$

$\Rightarrow y(n) \cong s(n)$ and $e(n) \cong v(n)$

Application: Adaptive Noise Cancellation



Domain:
Acoustics, Vibration, ...

Operation:

This is (unlike ALE) a 2-channel system

Primary sensor (e.g. MIC) is close to the signal source but also has noise parts

Secondary sensor is close to the noise source (measures a reference to the noise)

The noise components in the 2 sensors are not identical due to (e.g. acoustic) transfer function from noise source to sensors, but are correlated

The adaptive algorithm minimizes $e(n) = d(n) - y(n) = s(n) + x'(n) - y(n)$

Now $E[e^2(n)] = E[s^2(n)] + E[(x'(n) - y(n))^2] + 2E[s(n)x'(n)] - 2E[s(n)y(n)]$
 $= E[s^2(n)] + E[(x'(n) - y(n))^2]$

Minimization: $y(n) = x'(n) \Rightarrow W(z) = P(z) \Rightarrow e(n) = s(n)$ (note that here $e(n) \neq 0$)